Determinant of an n x n matrix

Note:

- I use Gaussian elimination with partial pivoting to make the matrix upper triangular (section 5.3 in the book).
- The determinant of an upper triangular matrix is the product of its diagonal elements.
- Subtracting a multiple of one row from another does not change the determinant.
- Each swap of two rows changes the sign of the determinant, so I count the swaps.
- Therefore, we have: det(A) = (-1)^(number of swaps) x (product of the diagonal elements after elimination).
- If the pivot is still 0 after pivoting, the whole column below the diagonal is 0. That makes the matrix is singular and det(A) = 0 because there is a 0 on the diagonal.

Determinant with numpy library

In [7]:
```
import numpy as np

# Use the given matrix in the exercise
A=[[1,2,1],
   [3,6,0],
   [3,5,0] ]
print("Determinant according to Numpy: {}".format(np.linalg.det(A)))
```

```
Determinant according to Numpy: -3.0000000000000004
```

Determinant with Gaussian elimination and partial pivoting

I will print my determinant and the one from numpy, so I can check my algorithm

In [8]:
```
# A: any n x n matrix
def determinant(A):
    U = np.array(A, dtype=float) #copy, so A is not changed
    n = U.shape[0]
    swaps = 0

    # Go through the columns 1 by 1 and make everything below the diagonal
0. The last column has nothing below the diagonal, so I stop at n-1
    for j in range(n - 1):
        # Partial pivoting: row with the largest |U[i, j]| from row j down
        p = j + np.argmax(np.abs(U[j:, j])) #argmax counts from row j, so I
add j to get the actual row number
        if p != j:
            U[[j, p]] = U[[p, j]] #swap row j and row p
            swaps += 1 #every swap flips the sign of the determinant

        # Pivot = 0 -> the largest value in the column is 0 -> the whole
column below is 0 -> nothing to eliminate and the 0 on the diagonal makes
det = 0
        if U[j, j] == 0:
            continue

        # Make the elements below the pivot 0, this does not change the
determinant
        for i in range(j + 1, n):
            m = U[i, j] / U[j, j] #multiplier, chosen so that U[i, j]
becomes 0
            # row i = row i - m * row j
            U[i, j:] -= m * U[j, j:] #start at column j because the columns
before j are already 0 in both rows


    # det = product of the diagonal, with the sign from the swaps
    det = (-1)**swaps * np.prod(np.diag(U))

    print("My determinant: ", det)
    print("Determinant according to Numpy: ", np.linalg.det(A))
    return det
```

In [9]:
```
det = determinant(A)
```

```
My determinant:  -3.0
Determinant according to Numpy:  -3.0000000000000004
```

Note:

- Without pivoting this matrix fails because when after the first column is eliminated, the second pivot is 0.
- My result is -3.0 Numpy gives -3.0000000000000004. The difference is because of floating point rounding.